A Extended Related Work

Budget-aware evaluation and test-time compute. Recent work argues for comparing methods at matched compute budgets and reporting token usage, and not just accuracy (Wang et al., 2024). Our protocol reports sequential budget B_{seq} (latency proxy along the accepted path) and total budget B_{total} (all tokens, including discarded branches), enabling apples-to-apples comparisons among single-pass, long-trace, sampling-heavy, and iterative pipelines.

MCTS, Learning to search and amortizing search. AlphaGo, AlphaZero, and MuZero couple a learned policy/value with an expensive test-time search (e.g., MCTS) that serves as an improvement operator; the outputs of search are then distilled back into the network, amortizing future search cost (Silver et al., 2016, 2017; Schrittwieser et al., 2020). *Expert Iteration* formalizes this loop as policy improvement via planning followed by supervised or RL updates toward the planner's targets (Anthony et al., 2017). Earlier "learning to search" work in structured prediction similarly alternates local rollouts with policy updates (e.g., SEARN, DAgger) (Daumé III & Langford, 2006; Ross et al., 2011). Our setting is analogous in spirit but distinct in mechanics: we operate in the *textual reasoning* regime with round-wise operators (**SR**, **PDR**) that keep the per-call sequential budget B_{seq} small, optionally raising total compute B_{total} via parallel drafts. Operator-consistent RL then amortizes this improvement procedure into the model weights.

Global workspace and modular coordination. Our compact, round-wise summary $C^{(r)}$ is conceptually related to the *shared global workspace* proposed by Goyal et al. (2022), which enables coordination among neural modules through a small communication bottleneck (inspired by Global Workspace Theory (Baars, 2005; Shanahan, 2006)). In contrast, our workspace is textual, re-synthesized at each round rather than persisted, and used as an inference-time operator. Thus, we borrow the coordination intuition while avoiding long-context replay and architectural changes.

B Prompts

B.1 Sequential Refinement

Refinement Prompt

```
Solve the following math problem efficiently and clearly. Please reason
    step by step, and put your final answer within $\\boxed{answer}$.

Where [answer] is just the final number or expression that solves the
   problem.

Problem: {{ problem }}

Here is an example candidate response wrapped in angle brackets:

<model_response>
# Solution response from previous turn
</model_response>

Treat the response as unverified; and come up with a better answer
   without starting from scratch.
```

B.2 Parallel-Distill-Refine

Refinement Prompt (Non-summary)

```
Solve the following math problem efficiently and clearly. Please reason
    step by step, and put your final answer within $\\boxed{answer}$.

Where [answer] is just the final number or expression that solves the
   problem.

Problem: {{ problem }}

Here are some candidate responses, each wrapped in angle brackets:

<model_response_1>
# Solution response from previous round
</model_response_1>

...

<model_response_k>
# Solution response from previous round
</model_response_k>

Treat these responses as unverified; and use these responses to come up
    with a better answer without starting from scratch.
```

Refinement Prompt (Summary)

```
Solve the following math problem efficiently and clearly. Please reason
    step by step, and put your final answer within $\\boxed{answer}$.

Where [answer] is just the final number or expression that solves the
   problem.

Problem: {{ problem }}

Here is a summary of the reasoning traces by a few other solvers:

<summary>
# Summary of all solutions from the previous iteration
</summary>

Treat the summary as unverified; and use the summary as context to come
    up with an answer.
```

B.3 Training Setup

We run a small SFT on the pre-trained 8B base model using a batch size of 2M tokens, max sequence length of 32768, and a learning rate of 2×10^{-5} using the AdamW optimizer (Loshchilov & Hutter, 2017) on 32 H100 GPU nodes for approximately 4 epochs and 8B tokens in total. For RL, we use a constant learning rate of 5×10^{-7}, AdamW optimizer (Loshchilov & Hutter, 2017) with $\epsilon = 10^{-15}$, weight decay of 0.01, and a linear warmup of 100 steps. We use 80 Nvidia H200 GPUs for the baseline RL run with a 64/16 generators/trainers split and 288 H200 GPUs for **PDR** and continual **PDR** RL with a 256/16 generators/trainers split to parallelize

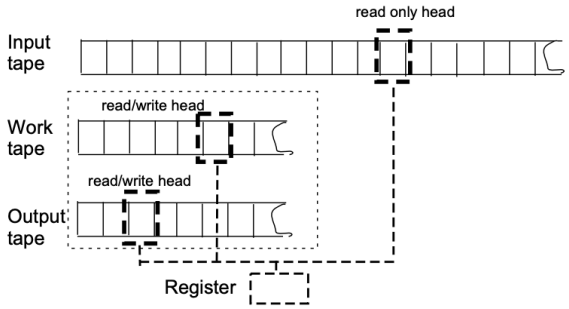

Figure 7 Space-bounded Turing Machine [Figure from *Computational Complexity* by Arora and Barak, 2007]. The input has size N and the machine has read-only capability for the input. A special "tape head" can be moved over the input to read bits from it. The amount of working memory (read/write/erase) for actual computation has size $S(N)$ where $S(N) \geq \log N$.

inference during rollout generation. We run all RL training for 800 steps. All evaluations are done with a temperature and `top-p` value of 1.0.

C Complexity of Space-bounded computation

Our work focused on language models that emit reasoning traces that are longer than the context size. We sketch similarity to the setting *space-bounded computation* which is formally studied in computational complexity theory. Since LLMs have probabilistic output (unless if temperature is set to 0) the fixed-context LLM considered in the paper is most similar to randomized space-bounded machine.

The most interesting result about randomized space-bounded machines is that if the input contains a graph of N vertices, then the randomized machine can determine connectivity of the N-vertex graph even though it only has $\mathcal{O}(\log N)$ space.

Furthermore, imagine that the graph of size N is a knowledge-graph whose local structure is known to the space-bounded machine. Specifically, given vertex indices i, j the space-bounded machine is able to determine whether edge $\{i, j\}$ exists in the graph. Then the machine does not need access to the full graph tape at all! It can do a random walk through the graph "in its mind" to determine connectivity. This is the closest setting to ours, whereby seemingly complex reasoning-connectivity of an N-node graph can be carried out in less than $\mathcal{O}(N)$ space.

D Additional Results

D.1 Oracle

Similar to Figure 6, we observe that having incorrect solutions in the context workspace can heavily degrade performance, and this effect is more noticeable for `o3-mini`.

D.2 SR and PDR operators

AIME 2025 results using the two iterative improvement operators **SR** and **PDR** are presented in Figure 9. For sequential token budget of $49k$, the performance on `o3-mini` improves from 73.5 for Long CoT to 77.1 using **SR** operator and further to 82.9 using the **PDR** operator.